BSIDES · ZACH GIBBS

Kubernetes
Orchestrating a More Secure Future

Live, interactive demos with real k3s clusters

SOURCE, SETUP, AND FULL DEMO ENVIRONMENT

https://github.com/AsocPro/bsides-k8s-2026

Clone the repo and run `dagger call present up --ports 8080:8080` to replay everything yourself.

Kubernetes

Orchestrating a More Secure Future

BSides 2026

Kube-Aid!
OH, YEAH!
DEVOPS REFRESHER!
ARTIFICALLY FLAVORED
CONTAINERIZED DRINK MIX
JUST ADD CLUSTERS!

How We Got Here

The evolution of application delivery

Bare Metal

Direct hardware install. Config drift, slow provisioning, inconsistent patching.

Virtual Machines

Snapshots & templates improved things. Still heavyweight, still mutable.

Containers

Immutable, lightweight, reproducible. Same image everywhere.

Kubernetes Core Concepts

Namespaces

Logical isolation boundaries for resources and policies. Think of them as folders that keep teams and environments separated.

Deployments

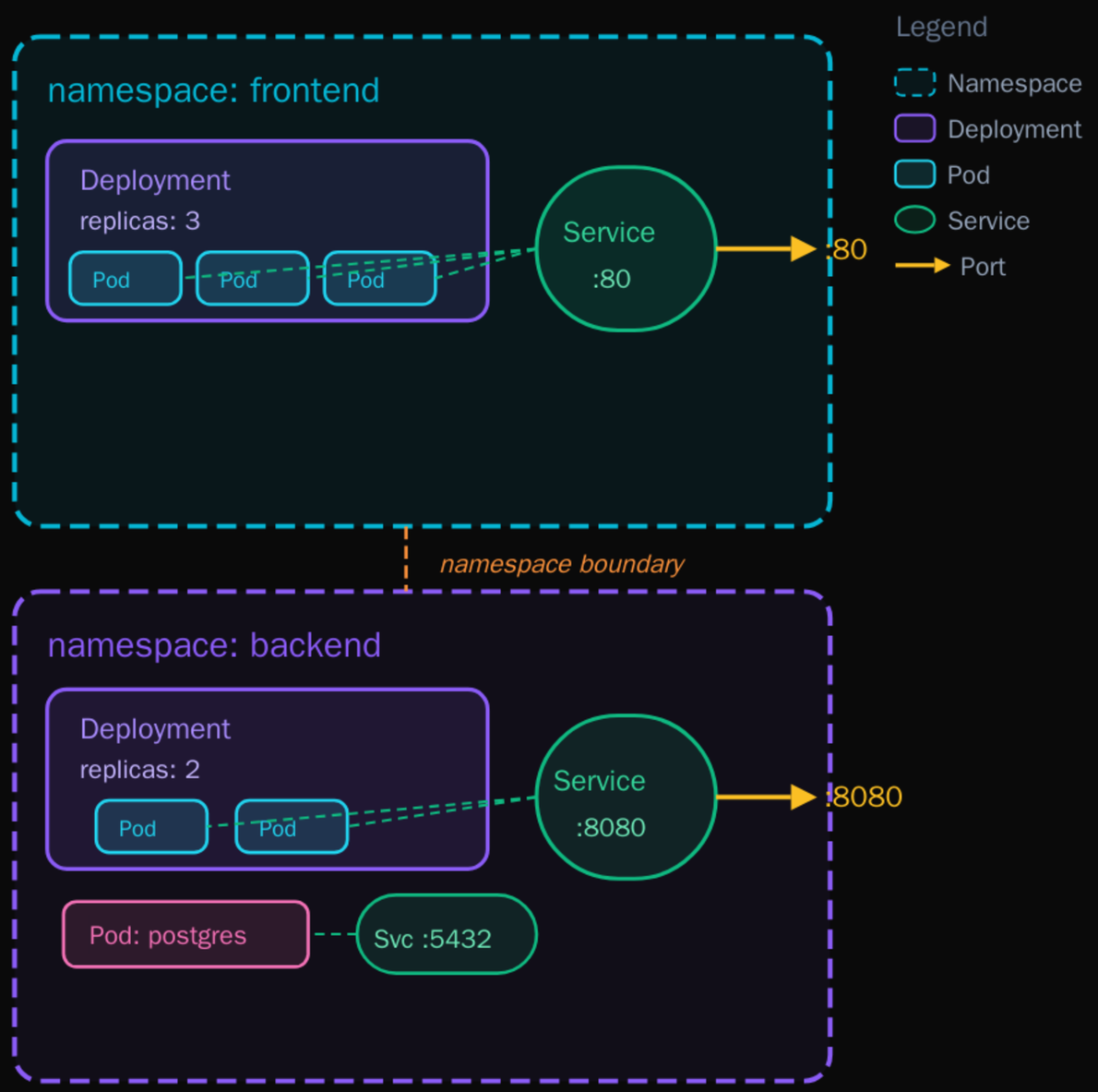
Declare your desired replica count and update strategy. Kubernetes ensures reality matches your intent.

Pods

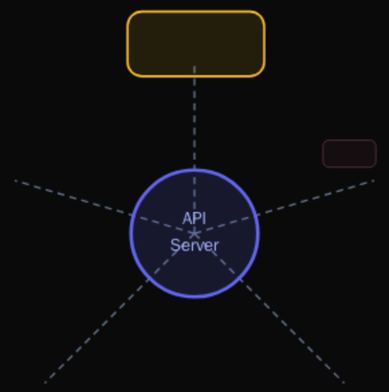
The smallest deployable unit — one or more containers sharing network and storage. Ephemeral by design.

Services

Stable networking abstraction that routes traffic to pods. Expose ports internally or externally.



Kubernetes Built-in Security



- All changes go through the API server — fully auditable
- Secrets management is a first-class concept
- Resource isolation via namespaces, quotas, and security contexts
- Security posture defined in code — reviewable and version controlled

RBAC

Making sure only the right people make changes

Roles & ClusterRoles

Define permissions on resources

```
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

RoleBindings

Associate roles with users or service accounts

```
kind: RoleBinding
metadata:
  name: read-pods
roleRef:
  kind: Role
  name: pod-reader
subjects:
- kind: ServiceAccount
  name: demo-user
```

Service Accounts

Identity for processes running in pods

```
kind: ServiceAccount
metadata:
  name: demo-user
  namespace: demo
```

Principle of Least Privilege

Grant only what's needed — no wildcards, no cluster-admin

RBAC Demo

Live — try creating roles and testing access

Check State | Monitor

Step 1
Try listing pods with no permissions

Step 2
Create a Role allowing pod access

Step 3
Bind the role to demo-user

Step 4
Retry — should now succeed

Step 5
Try deleting — least privilege in action

RBAC

Live terminal – see README for how to run the full demo locally.

Policy Agents

Making sure the changes are reasonable

RBAC controls who can make changes.

Policy agents control what changes are allowed.

> *Enforced via admission webhooks before resources hit the cluster.*

Policies You Can Enforce

- Require all containers to run as non-root
- Block privileged containers
- Enforce resource limits (CPU/memory) on all pods
- Mandate images come from approved registries only
- Prevent use of latest tag — require explicit versioning

Policy Agent Demo

Live — Kyverno blocking non-compliant resources

Check State | Monitor

Step 1
Apply Kyverno policies

Step 2
Inspect a policy

Step 3
Try deploying a root container — rejected

Step 4
Deploy a non-root container — accepted

Step 5
Test registry policy with unapproved image

POLICY

Live terminal – see README for how to run the full demo locally.

Network Policies

Limiting blast radius when something is exploited

By default, all pods can communicate with each other. Network policies implement microsegmentation — explicit allow-lists for traffic between pods.

- Microsegmentation — each service only talks to its direct dependencies
- Defense in depth — network controls remain even if app controls fail
- Reduced lateral movement — attackers can't pivot from frontend to database
- Compliance — demonstrate network isolation for PCI-DSS, SOC2

Network Policy Demo

Live — microsegmentation in action

Check State | Monitor

Step 1
Show the running 3-tier app

Step 2
Everything can talk to everything — the problem

Step 3
Apply default-deny — everything goes dark

Step 4
Restore allowed paths one by one

Step 5
Frontend → database still blocked — blast radius contained

NETPOL

Live terminal – see README for how to run the full demo locally.

Looking Forward

The next generation of secure deployments

eBPF / Cilium

Kernel-level security — L7 network policies, runtime detection, zero overhead.

Workload Identity

Short-lived credentials, automatic rotation, no static secrets to leak.

Talos Linux

No SSH, no shell, no userspace. Minimal attack surface by design.

SPIFFE / SPIRE

Universal workload identity — cryptographic proof of who is talking to whom.

Questions?

Zach Gibbs

Scan for session feedback